

COVIS Lab 3: Deep Learning

Centrale Nantes

1 Introduction

In this lab, we will train a simple Multi-Layer Perceptron (MLP) to classify images of handwritten digits in the [MNIST](#) dataset using the deep learning framework [PyTorch](#).

1.1 MNIST Dataset

The MNIST dataset contains images of single digits (from 0 to 9) written by humans as shown in Figure 1. These images have a size of 28×28 pixels and contain a single channel (i.e., grayscale).

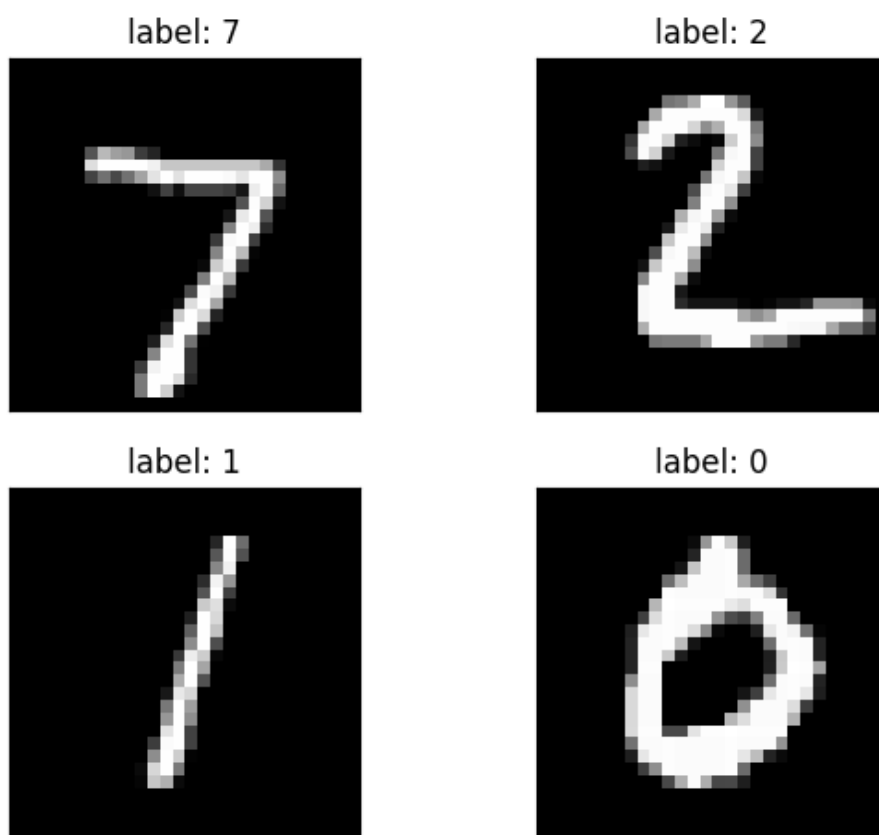


Figure 1: Sample images from the MNIST dataset

This dataset contains 60,000 data samples for training and 10,000 data samples for testing. Each data sample is a pair of **image** and **label**.

1.2 Model

1.2.1 Architecture

The MLP we will train in this lab has the architecture shown in Figure 2. This MLP consists of two Linear (a.k.a. fully connected or dense) layers. The first layer receives input of size $(N, 28^2)$ and maps it to a tensor of size (N, N_HIDDEN) . The second layer takes the output of the first layer and maps it to another tensor of size $(N, 10)$. Here, N is the number of data samples that are fed into the model, and 10 is the number of classes in MNIST.

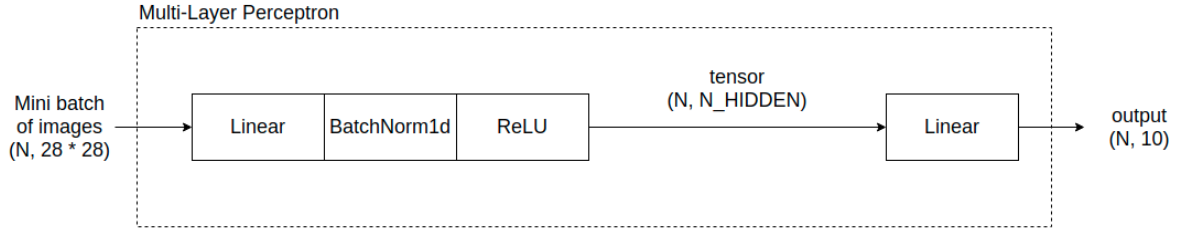


Figure 2: Architecture of an MNIST classifier

The output of the model (a tensor of size $(N, 10)$) is interpreted as follows: element j of row i represents the probability that image i belongs to class j .

The first Linear layer is followed by a BatchNorm layer to accelerate the training process. The theory behind this layer is outside the scope of this lab. Interested readers can refer to this [blog](#) or the [BatchNorm paper](#) for more details. To help the model learn non-linear patterns, the output of the BatchNorm layer is activated by the ReLU function defined in Equation 1:

$$\text{ReLU}(x) = \max(0, x) \quad (1)$$

Since the output of the last Linear layer is an unbounded float value (in the range of $(-\infty, +\infty)$), it is not yet a class probability (which is a float value in $[0, 1]$). This unbounded value is called a **logit** and can be mapped into a probability using the Softmax function defined as follows:

Let $\mathbf{x} = [x_0, x_1, \dots, x_{n-1}]$ be a vector of unbounded float values (i.e., $x_i \in (-\infty, +\infty)$)

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=0}^{n-1} \exp(x_j)} \quad (2)$$

The Softmax function is applied to each row of the MLP's output (size $(N, 10)$) to obtain another tensor of size $(N, 10)$. Row i of the resulting tensor contains elements within the range of $[0, 1]$, thus representing the class probabilities for image i .

1.2.2 Loss Function

The MLP is trained by minimizing the cross-entropy loss. Assuming $\mathbf{x} = [x_0, x_1, \dots, x_9]$ is the model's predicted class **probability** for an image I which has the label l (where l is the digit that I represents), the cross-entropy loss for the data sample (I, l) is:

$$\text{cross_entropy}(\mathbf{x}, l) = -\log(x_l) \quad (3)$$

This equation represents the cross-entropy loss for a single sample, where $\mathbf{x}$ is the vector of predicted probabilities after applying the Softmax function, and x_l is specifically the predicted probability for the correct class indexed by label l . This is also known as the negative log-likelihood. For multi-class classification problems like MNIST, minimizing this loss encourages the model to assign higher probability to the correct class.

In practice, at each training step, several data samples (a mini-batch) are fed into the model and the cross-entropy loss is computed for all of these samples. The average value is then used as the final loss.

Intuitively, minimizing the cross-entropy loss forces the model to increase the predicted probability of the correct class. Since this probability is produced by the Softmax function, increasing the probability of the correct class effectively lowers the probability of the incorrect classes.

2 Implementation

2.1 Training Loop

The training pipeline is illustrated in Figure 3. At each training step, a mini-batch of data (a set of image-label pairs) is taken from the dataset and fed to the model. The model performs its prediction. Then, the loss function uses the labels and the model's predictions to compute a scalar value called the loss, which quantifies how poorly the model performed on this mini-batch. This loss is then propagated backward from the last layer to the first layer of the model to adjust the model's parameters in a way that minimizes the loss function.

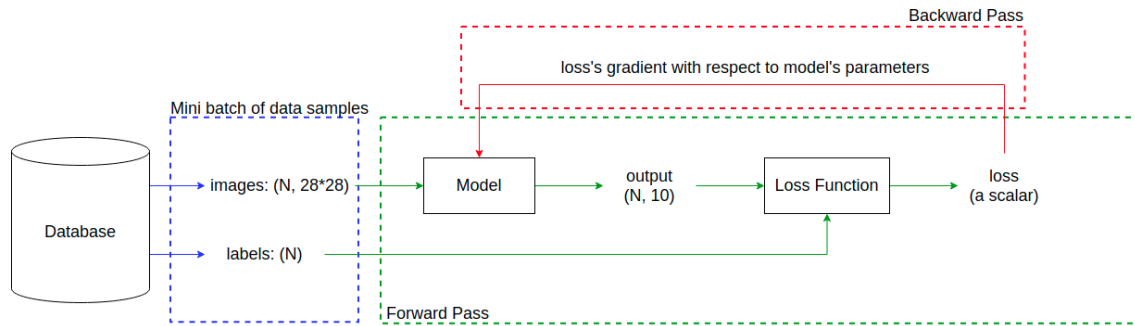


Figure 3: Training pipeline

In the PyTorch framework, the backward pass is automatically defined once the model's forward pass is defined. With the backward pass already taken care of, our tasks are:

1. Build the dataset from raw data
2. Define the model and loss function
3. Define the training loop
4. Get a coffee while the model is being trained
5. Evaluate the model's performance

The training loop and the evaluation are implemented in `train.py` and `eval_classifier.py` respectively. No modification is required for them unless you want to try your own training loop or add new metrics for evaluation. To complete this lab, you need to do steps 1 and 2, which involve building the dataset and defining the model.

2.2 Expected Work

2.2.1 Dataset

The dataset is defined by the class `MNISTDataset` in `dataset.py`, where you'll need to fill in a few blanks. The objectives of this script are:

1. Parse raw data and store it in a database.
2. Provide indexing into the database, which is a function that accepts an index (an integer) as input and returns a data sample - a pair of (image, label). This is carried out by the `__getitem__` method, which you need to fill in.
3. Provide a function for batching individual data points - pairs of (image, label) - into a mini-batch of data (a tensor storing N images and a tensor storing N labels). This is carried out by the `collate_func` method, which you need to fill in.

2.2.2 Model

The model is defined by the class `MNISTClassifier` in `model.py`, where you need to fill in some blanks.

The constructor of this class (i.e., its `__init__` method) defines the layers that make up the model and stores these layers in the attribute `self.net`. In addition, the loss function (cross-entropy) is stored in the attribute `loss_fnc`.

The forward pass of the model is defined by the `forward` method, which accepts a mini-batch in the form of a Dictionary as input and returns another Dictionary as output. The output of this method depends on the operation mode of the model (training or evaluating). If the model is in training mode, its loss function is invoked to compute the loss. On the other hand, in evaluation mode, the `decode_prediction` method is invoked to compute class probabilities from logits and produce class predictions as the classes with the highest probabilities.

You are required to complete the `__init__` method to finish the model definition and implement the `decode_prediction` method to decode the model's predictions.

2.2.3 Training

Once you finish `dataset.py` and `model.py`, you can invoke `train.py` by opening a terminal in the directory containing `train.py` and typing "python train.py 128" (where 128 represents the `N_HIDDEN` parameter, which can be set to any value you wish). The training and subsequent evaluation will then take place.

2.2.4 Environment Setup

Before starting the lab, you need to set up the Python environment with all the required dependencies. You can use the following commands in a terminal to create and configure a Conda environment.

If you do not have conda installed (anaconda or miniconda), just run the 'pip install' line.

```
conda create -n covis_lab_env python=3
conda activate covis_lab_env
pip install tensorboardX torch matplotlib python-mnist tqdm
```

After running these commands, your environment will be ready with all the necessary packages for this lab.